DAY 4 · TERRAFORM STATE

This one file can break your whole infra team.

✗ local only

✗ no locking



✗ in Git

✗ secrets inside

terraform.tfstate

DEJI AJISHE · DevOps | Multi-Cloud

How to protect it →

INFOGRAPHIC 1/3 TERRAFORM STATE MANAGEMENT

The process every Terraform run follows with state

State maps your code to real resource IDs, attributes and dependencies. It can hold secrets in plain text. Know each step and you know where things break.

01

Lock

The backend takes a lock before anything is read or written.

Why: two applies at once can corrupt the state file.

```
terraform plan -lock-timeout=5m   # wait for a lock instead of failing
```

02

Refresh

Terraform reads the real resources and updates its view of them.

Why: manual console changes (drift) surface here, before any diff.

```
terraform plan -refresh-only   # show drift, change nothing
```

03

Plan

Code is compared with the refreshed state; the result is a saved plan file.

Why: a saved plan means what was reviewed is exactly what gets applied.

```
terraform plan -out=tfplan
```

04

Apply

Changes are made in dependency order, from the reviewed plan only.

Why: applying without a plan file can pick up changes nobody reviewed.

```
terraform apply tfplan
```

05

Write a new state version

The serial number goes up; the lineage ID stays the same.

Why: serial and lineage stop an older or unrelated file overwriting a newer one.

```
terraform state pull | jq '.serial, .lineage'
```

06

Unlock

The lock is released automatically when the run ends.

Why: a crashed run can leave a stale lock. Force-unlock only when no run is active.

```
terraform force-unlock LOCK_ID
```

INFOGRAPHIC 2/3 REMOTE STATE SETUP

Set up remote state safely in AWS, Azure and GCP

07

Create isolated storage per environment

A dedicated bucket or storage account for each environment, in an admin account or project.

Why: one compromised environment can't read or overwrite another's state.

```
aws s3api create-bucket --bucket acme-tfstate-prod --region us-east-1
```

08

Harden it: versioning, encryption, no public access

Every previous state stays recoverable; nothing is reachable from the internet.

Why: versioning is your undo button when state is lost or corrupted.

```
gcloud storage buckets update gs://acme-tfstate-prod --versioning
```

09

Point the backend at it, with locking on

One key (path) per environment and component.

Why: small states mean fast plans and a small blast radius.

```
terraform {
  backend "s3" {
    bucket       = "acme-tfstate-prod"
    key          = "network/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true   # S3 native locking, Terraform 1.10+
  }
}
```

10

Migrate and verify

Move the existing local state into the backend, then confirm nothing changed.

Why: an empty plan after migration proves Terraform still owns everything.

```
terraform init -migrate-state && terraform plan
```

	AWS	Azure	Google Cloud
Storage	S3 bucket	Storage account + blob container	Cloud Storage bucket
Locking	use_lockfile (.tflock object)	Blob lease, built in	Lock file, built in
Encryption	SSE-S3 by default; SSE-KMS for your own key	On by default; customer-managed keys optional	On by default; Cloud KMS keys optional
Pipeline access	IAM role scoped to the bucket and key	Storage Blob Data Contributor on the container	Storage Object Admin on the bucket

INFOGRAPHIC 3/3 CHANGE, RECOVER, PROTECT

Change and recover state without incidents

11

Back up before any state change

Take a copy before refactors, imports, upgrades or force-unlocks.

Why: the cheapest insurance in Terraform; restoring takes minutes.

```
terraform state pull > backup-$(date +%F).tfstate
```

12

Refactor with code, not state surgery

moved (1.1+) renames, import (1.5+) adopts, removed (1.7+) stops managing without deleting.

Why: the change is reviewed in a plan like any other code.

```
moved {
  from = aws_instance.web
  to   = aws_instance.app
}
```

13

Detect drift on a schedule

Run a refresh-only plan nightly in CI and alert when it finds changes.

Why: drift found early is a small fix, not an outage during the next apply.

```
terraform plan -refresh-only -detailed-exitcode   # exit 2 = drift
```

14

Recover from a bad state version

Find the last good version, restore it, then plan and check for surprises.

Why: versioning only helps if you've practised the restore.

```
aws s3api list-object-versions --bucket acme-tfstate-prod --prefix network/
```

✓ Must know

- **sensitive = true** hides values in CLI output, not in state
- **terraform_remote_state** exposes the whole state; publish outputs instead
- **One writer:** only the pipeline role can write state
- ***.tfstate** belongs in .gitignore, always

✗ Pitfalls that cause incidents

- Local state on one engineer's laptop
- Applying from a laptop while CI also runs
- One giant state for every environment
- Force-unlocking a run that is still active

WHAT & WHY

What Terraform state is, and why it can hurt you

State is Terraform's record of which real resources belong to which configuration: resource IDs, attributes, dependencies and metadata. Every plan compares your code with state and with the real cloud.

It contains

Resource addresses and IDs · attribute values · outputs · dependency data · a serial number and lineage ID

It can also contain

Database passwords, keys and other sensitive values **in plain text**, even when marked sensitive

Why it matters: lose it and Terraform forgets what it manages. Corrupt it and plans lie. Leak it and secrets leak.

HOW IT WORKS

What happens to state during plan and apply

01 **Lock:** the backend takes a lock so no one else can write at the same time

02 **Refresh:** Terraform reads the real resources and updates its view of them

03 **Diff:** compares code with the refreshed state and builds a plan

04 **Apply:** makes the changes in dependency order

05 **Write:** stores a new state version with an incremented serial

06 **Unlock:** releases the lock for the next run

Lineage and serial stop an older or unrelated state file from silently overwriting a newer one.

REFERENCE ARCHITECTURE

A safe remote-state setup

CI runner, not a laptop

Short-lived cloud credentials via OIDC. One job per root configuration.

Locking

S3 native lockfile · Azure blob lease · GCS lock object. A second run waits or fails.

Versioned, encrypted bucket

Versioning on, so every previous state is recoverable. Encryption at rest, optionally with your own key. Public access blocked.

One state per root

Split by environment and component: e.g. prod/network, prod/data, prod/app.

Least-privilege access

Only the pipeline role can write. Very few people can read (state can hold secrets).

THE PROCESS

Setting up state the right way

01 **Create a dedicated bucket or storage account** per environment, in a separate admin account/project

02 **Turn on versioning, encryption and public-access blocking**

03 **Restrict access:** pipeline role writes; break-glass humans read

04 **Configure the backend** with locking in each root

05 **Migrate existing local state:** terraform init -migrate-state

06 **Split large states** by environment and component

07 **Run plan/apply only from CI**, one job per root

08 **Practice a restore** from a previous state version

STEPS 1–2 · BOOTSTRAP THE BACKEND

Create the state storage in each cloud

```
# AWS: S3 bucket (new buckets are encrypted by default)
aws s3api create-bucket --bucket acme-tfstate-prod --region us-east-1
aws s3api put-bucket-versioning --bucket acme-tfstate-prod \
  --versioning-configuration Status=Enabled
aws s3api put-public-access-block --bucket acme-tfstate-prod \
  --public-access-block-configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,Restrict
PublicBuckets=true

# Azure: storage account + container + blob versioning
az storage account create -n acmetfstateprod -g rg-tfstate -l eastus \
  --sku Standard_ZRS --min-tls-version TLS1_2 \
  --allow-blob-public-access false
az storage container create -n tfstate \
  --account-name acmetfstateprod --auth-mode login
az storage account blob-service-properties update -n acmetfstateprod \
  -g rg-tfstate --enable-versioning true

# GCP: bucket with uniform access, no public access, versioning
gcloud storage buckets create gs://acme-tfstate-prod \
  --location=us-central1 \
  --uniform-bucket-level-access --public-access-prevention
gcloud storage buckets update gs://acme-tfstate-prod --versioning
```

STEP 4 · CONFIGURE THE BACKEND

Backend blocks with locking

```
# AWS: native S3 locking (Terraform 1.10+), replacing DynamoDB locks
terraform {
  backend "s3" {
    bucket       = "acme-tfstate-prod"
    key          = "network/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
# Azure: blob lease locking; Entra ID auth instead of account keys
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "acmetfstateprod"
    container_name       = "tfstate"
    key                  = "network.tfstate"
    use_azuread_auth     = true
  }
}
# GCP: locking built into the gcs backend
terraform {
  backend "gcs" {
    bucket = "acme-tfstate-prod"
    prefix = "network"
  }
}
```

STEPS 5–6 · MIGRATE AND SPLIT

Move and reshape state without downtime

```
# Move local state into the new remote backend
terraform init -migrate-state

# See what a state manages
terraform state list

# Take a backup before any surgery
terraform state pull > backup-$(date +%F).tfstate
```

```
# Rename or move a resource without destroy/recreate (TF 1.1+)
moved {
  from = aws_instance.web
  to   = aws_instance.app
}
# Adopt an existing resource into state (TF 1.5+)
import {
  to = aws_s3_bucket.logs
  id = "acme-logs"
}
# Stop managing a resource without destroying it (TF 1.7+)
removed {
  from = aws_s3_bucket.legacy
  lifecycle { destroy = false }
}
```

STEP 7 · ORCHESTRATION IN CI

Running state-safe pipelines

```
# Wait for a lock instead of failing instantly
terraform plan  -lock-timeout=5m -out=tfplan -input=false
terraform apply -lock-timeout=5m -input=false tfplan
```

→ **One pipeline job per root**, with CI-level concurrency limits so two runs of the same root never overlap

→ **Saved plans only:** apply exactly what was reviewed

→ **Separate roles** for plan (read) and apply (write) where possible

→ **Never run apply from a laptop** against shared state; local runs use plan only

→ **Alert on long-held locks:** a stuck lock usually means a crashed run

STEP 8 · DRIFT AND RECOVERY

Detect drift, recover safely

```
# Detect drift without changing anything
terraform plan -refresh-only

# Accept real-world changes into state (after review)
terraform apply -refresh-only

# Find an earlier state version (AWS example)
aws s3api list-object-versions --bucket acme-tfstate-prod \
  --prefix network/terraform.tfstate

# Last resort: release a lock left by a crashed run
terraform force-unlock LOCK_ID
```

Before force-unlock: confirm no run is still active. Unlocking a live run is how state gets corrupted.

SHARING DATA BETWEEN STATES

terraform_remote_state, or publish outputs?

terraform_remote_state

Simple, but the reader needs access to the other config's **entire state**, secrets included

Publish specific outputs

Write just the values consumers need to a parameter store, then read them with a data source

```
# Producer (network root) publishes one value
resource "aws_ssm_parameter" "vpc_id" {
  name  = "/prod/network/vpc_id"
  type  = "String"
  value = aws_vpc.this.id
}
# Consumer (app root) reads only that value
data "aws_ssm_parameter" "vpc_id" {
  name = "/prod/network/vpc_id"
}
```

Azure: App Configuration or Key Vault · GCP: Secret Manager or direct data sources

SECURITY

Treat state like a password vault

- → **sensitive = true hides values in CLI output**; it does **not** encrypt them in state
- → **Encrypt at rest**, ideally with a customer-managed key (KMS / Key Vault / Cloud KMS)
- → **Restrict reads** as tightly as writes; reading state can mean reading secrets
- → **Never commit state**: add *.tfstate and *.tfstate.* to .gitignore
- → **Keep secrets out of state** where possible: reference a secret manager instead of passing values
- → **Newer tooling helps:** recent Terraform versions add ephemeral values; OpenTofu supports client-side state encryption
- → **Audit access** to the state bucket with cloud audit logs

PITFALLS

8 state mistakes that cause incidents

- → **Local state** on one engineer's laptop
- → **State committed to Git**, secrets and all
- → **No locking:** two applies corrupt the same file
- → **One giant state**: slow plans, everything at risk on every change
- → **Versioning off:** a bad write can't be undone
- → **Broad read access** to state buckets
- → **Force-unlocking a live run**
- → **Hand-editing state** instead of using moved, import and removed

NEED-TO-KNOWS

What every Terraform engineer should know about state

→ S3 backend locking now works **natively with use_lockfile** (Terraform 1.10+)

→ Azure locks state with **blob leases**; GCS uses a **lock object**

→ **-refresh-only** shows drift without proposing config changes

→ **moved** (1.1+), **import** (1.5+) and **removed** (1.7+) blocks make refactors reviewable

→ **Workspaces** share a backend with separate state files; separate roots are clearer for prod

→ **terraform state pull** is the quickest backup before risky work

→ A state's **serial** increments on every write; **lineage** identifies it

→ **Sensitive ≠ encrypted**: protect the storage itself

CHECKLIST

Terraform state checklist

- ☐ Remote backend for every root; no local state anywhere
- ☐ Versioning, encryption and public-access blocking on
- ☐ Locking enabled; lock-timeout set in CI
- ☐ Write access only for the pipeline role; reads tightly limited
- ☐ State split by environment and component
- ☐ *.tfstate in .gitignore
- ☐ Outputs shared via a parameter store, not whole-state reads
- ☐ Refactors use moved / import / removed blocks
- ☐ Drift checked on a schedule with -refresh-only
- ☐ A restore from a previous version has been practiced

HANDS-ON LAB

Try it: Bootstrap, migrate and restore state

01 Create a versioned, encrypted state bucket in a sandbox (bootstrap page)

02 Point a small config at it with a backend block and migrate

03 Run two plans at once and watch the second one wait for the lock

04 Restore an earlier state version and confirm plan shows no surprises

```
terraform init -migrate-state          # move local state to the bucket
terraform state pull > backup.tfstate  # take a backup
terraform plan -lock-timeout=5m        # confirm locking works
# change something, apply, then find the previous version:
aws s3api list-object-versions --bucket acme-tfstate-dev \
  --prefix network/terraform.tfstate
```

> **Teardown:** terraform destroy the test resources, then delete the sandbox bucket (all versions).

FURTHER READING

Go deeper: official references

HashiCorp: State · S3 backend
developer.hashicorp.com/terraform/language/state · /backend/s3

AWS Prescriptive Guidance: Backend best practices
docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/backend.html

Microsoft: Store Terraform state in Azure Storage
learn.microsoft.com/en-us/azure/developer/terraform/store-state-in-azure-storage

Google Cloud: Store Terraform state in Cloud Storage
cloud.google.com/docs/terraform/resource-management/store-state

HashiCorp: Refactoring (moved) · Import blocks
developer.hashicorp.com/terraform/language/modules/develop/refactoring

Docs change. Always check the current version before you design.

YOUR TURN

Where does your Terraform state live today: S3, Azure Blob, GCS or HCP Terraform?

Share your safeguards in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 4 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a